Java Collection Framework

1. Introduction to the Java Collection Framework

The Java Collection Framework (JCF) is a set of interfaces, classes, and algorithms provided by Java to store, retrieve, manipulate, and process groups of objects efficiently.

It is available mainly in the `java.util` package.

Before the Collection Framework, Java programmers often used arrays to store multiple values. However, arrays have a fixed size, whereas many collection classes can grow or shrink dynamically.

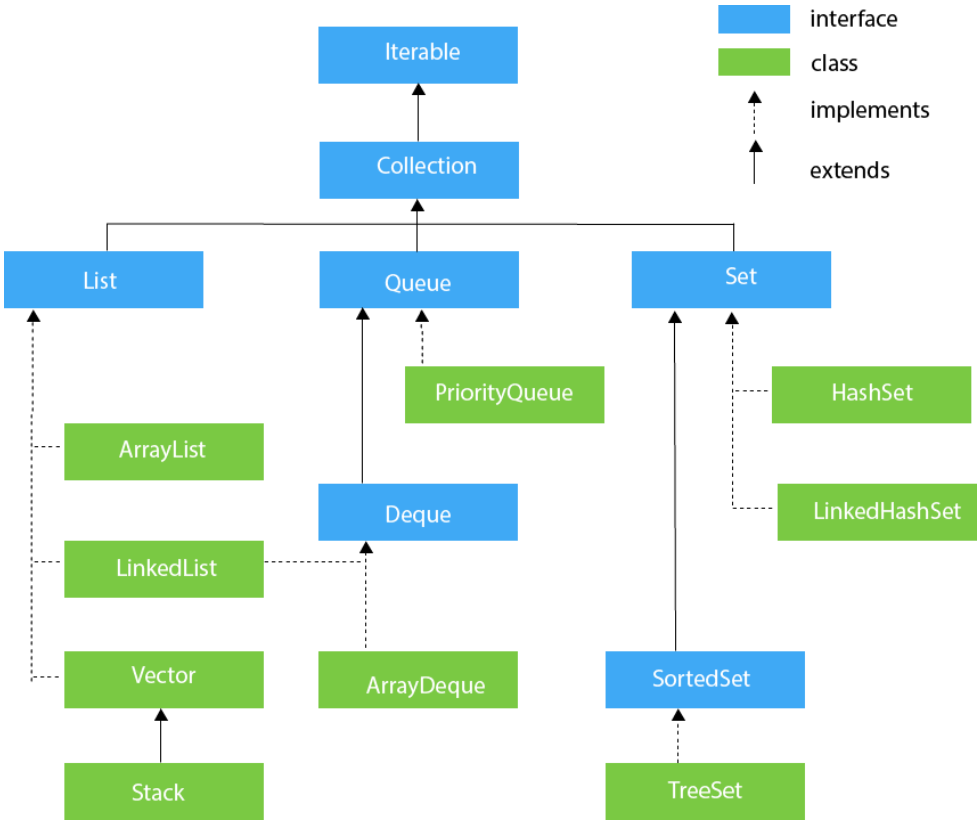
Why do we need the Collection Framework?

- To store multiple objects in a single data structure.
- To insert, delete, search, and update elements easily.
- To manage data dynamically without specifying a fixed size.
- To provide reusable data structures and standard methods.
- To reduce programming effort and improve code readability.

Arrays vs. Collections

Feature	Array	Collection
Size	Fixed	Usually dynamic
Data storage	Primitives and objects	Objects; wrapper classes for primitives
Insertion/deletion	Often requires manual shifting	Methods are available
Built-in operations	Limited	Many predefined methods
Example	`int[] a = new int[5];`	`ArrayList<Integer> list = new ArrayList<>();`

2. Hierarchy of the Java Collection Framework



Note: `Map` is also part of the Java Collections Framework, but it does not extend the `Collection` interface. Examples include `HashMap`, `TreeMap`, and `LinkedHashMap`.

3. Introduction to the List Interface

The `List` interface represents an ordered collection of elements.

Its main characteristics are:

1. Elements maintain a defined order.
2. Duplicate elements are allowed.
3. Elements can be accessed using their index.
4. Elements can be inserted or removed at specific positions.
5. Lists can contain `null` values, depending on the implementation.

Example:

```
import java.util.*;

public class ListExample {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
```

```
        names.add("Aman");
        names.add("Riya");
        names.add("Aman");

        System.out.println(names);
    }
}
```

Output:

```
[Aman, Riya, Aman]
```

The duplicate value `"Aman"` is allowed, and the insertion order is maintained.

4. Introduction to the ArrayList Class

The `ArrayList` class is a resizable-array implementation of the `List` interface.

It belongs to the `java.util` package.

Important characteristics

- Its size can increase or decrease dynamically.
- It maintains insertion order.
- It allows duplicate elements.
- It supports index-based access.
- It permits multiple `null` elements.
- It is not synchronized by default.
- Accessing an element by index is generally \(O(1)\).
- Inserting or removing an element in the middle is generally \(O(n)\), because elements may need to be shifted.

Creating an ArrayList

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        System.out.println(numbers);
    }
}
```

Output:

```
[10, 20, 30]
```

Here, `Integer` is used instead of `int` because Java collections store objects rather than primitive values.

5. Important Methods of the ArrayList Class

The following methods are frequently used in Java programming.

Method	Description	Example
`add(element)`	Adds an element at the end	`list.add(40);`
`add(index, element)`	Inserts an element at a position	`list.add(1, 15);`
`get(index)`	Returns an element at an index	`list.get(0);`
`set(index, element)`	Replaces an existing element	`list.set(0, 50);`
`remove(index)`	Removes an element by index	`list.remove(1);`
`remove(object)`	Removes a matching element	`list.remove(Integer.valueOf(20));`
`size()`	Returns the number of elements	`list.size();`
`contains(element)`	Checks whether an element exists	`list.contains(30);`
`indexOf(element)`	Returns the first matching index	`list.indexOf(20);`
`lastIndexOf(element)`	Returns the last matching index	`list.lastIndexOf(20);`
`isEmpty()`	Checks whether the list is empty	`list.isEmpty();`
`clear()`	Removes all elements	`list.clear();`
`addAll(collection)`	Adds all elements of another collection	`list.addAll(other);`
`removeAll(collection)`	Removes elements also present in another collection	`list.removeAll(other);`
`retainAll(collection)`	Keeps only common elements	`list.retainAll(other);`

`toArray()`	Converts the list to an array	`list.toArray();`
`subList(from, to)`	Returns a view of a range of elements	`list.subList(1, 3);`

Important: ArrayList indexes start at `0`. For example, the first element is at index `0`, and the last element is at index `size() - 1`.

6. Demonstration of ArrayList Methods

```
import java.util.ArrayList;

public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();

        // 1. Add elements
        list.add(10);
        list.add(20);
        list.add(30);
        System.out.println("Original: " + list);

        // 2. Insert at a specific index
        list.add(1, 15);
        System.out.println("After insertion: " + list);

        // 3. Access an element
        System.out.println("Element at index 2: "
                           + list.get(2));

        // 4. Update an element
        list.set(0, 5);
        System.out.println("After update: " + list);

        // 5. Remove an element by index
        list.remove(2);
        System.out.println("After removal: " + list);

        // 6. Search for an element
        System.out.println("Contains 30: "
                           + list.contains(30));

        // 7. Find the size
        System.out.println("Size: " + list.size());

        // 8. Check whether it is empty
        System.out.println("Is empty: " + list.isEmpty());

        // 9. Traverse the list
        for (Integer value : list) {
            System.out.println(value);
        }
```

```
        // 10. Remove all elements
        list.clear();
        System.out.println("After clear: " + list);
    }
}
```

Expected output

```
Original: [10, 20, 30]
After insertion: [10, 15, 20, 30]
Element at index 2: 20
After update: [5, 15, 20, 30]
After removal: [5, 15, 30]
Contains 30: true
Size: 3
Is empty: false
5
15
30
After clear: []
```

7. Traversing an ArrayList

There are several ways to access all elements of an ArrayList.

Method 1: Using a for loop

```
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}
```

Method 2: Using an enhanced for loop

```
for (Integer element : list) {
    System.out.println(element);
}
```

Method 3: Using an Iterator

```
import java.util.Iterator;

Iterator<Integer> itr = list.iterator();

while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

Method 4: Using forEach and a lambda expression

```
list.forEach(element -> System.out.println(element));
```

The enhanced for loop is straightforward for reading elements, while an `Iterator` is useful when you need to remove elements safely during traversal.

8. Practical Example: Managing Student Marks

```
import java.util.ArrayList;

public class StudentMarks {
    public static void main(String[] args) {
        ArrayList<Integer> marks = new ArrayList<>();

        marks.add(75);
        marks.add(82);
        marks.add(91);
        marks.add(68);

        int total = 0;

        for (int mark : marks) {
            total += mark;
        }

        double average = (double) total / marks.size();

        System.out.println("Marks: " + marks);
        System.out.println("Total: " + total);
        System.out.println("Average: " + average);
        System.out.println("Highest mark: "
                           + java.util.Collections.max(marks));
    }
}
```

Output:

```
Marks: [75, 82, 91, 68]
Total: 316
Average: 79.0
Highest mark: 91
```

This example demonstrates how an ArrayList can store a variable number of student marks and support calculations using built-in methods and loops.

9. Important points for examinations

- Collection Framework: A set of interfaces and classes for managing groups of objects.
- List: An ordered collection that allows duplicates.
- ArrayList: A resizable-array implementation of the `List` interface.
- `add()` vs. `set()`: `add()` inserts a new element, whereas `set()` replaces an existing element.
- `remove(1)` vs. `remove(Integer.valueOf(1))`: The first removes the element at index `1`; the second removes the first occurrence of the value `1`.

- `size()` vs. array `length`: Collections use `size()`, whereas arrays use the `length` property.
- `clear()` vs. `remove()`: `clear()` removes all elements; `remove()` removes a specified element or position.

In summary: The Java Collection Framework provides reusable data structures, and `ArrayList` is especially useful when a program requires an ordered, dynamically sized collection with frequent index-based access.